

JavaScript笔记

2026年7月10日 15:36

标识符

标识符 (identifier) 指的是用来识别各种值的合法名称。最常见的标识符就是变量名

标识符是由: 字母、美元符号(\$)、下划线(_)和数字组成, 其中数字不能开头

温馨提示

中文是合法的标识符, 可以用作变量名 (不推荐)

变量提升

JavaScript 引擎的工作方式是, 先解析代码, 获取所有被声明的变量, 然后再一行一行地运行。这造成的结果, 就是所有的变量的声明语句, 都会被提升到代码的头部, 这就叫做变量提升 (hoisting)。

```
1 console.log(num);
2 var num = 10; // 结果是什么呢?
```

1. 以下哪种不是JS文件引入到HTML文件中的方式:

- ☒ A JS嵌入到HTML文件中
- ☐ B 引入本地独立JS文件
- ☐ C 引入网络来源的JS文件
- ☐ D JS嵌入到CSS文件中

JavaScript输出方式

JavaScript有很多种输出方式, 都可以让我们更直观的看到程序运行的结果

```
1 // 在浏览器中弹出一个对话框, 然后把要输出的内容展示出来, alert都是把要输出的内容首先转换为
  字符串然后在输出的
2 alert("要输出的内容");
3 |
4 document.write("要输出的内容");
5
6 // 在控制台输出内容
7 console.log("要输出的内容");
```



数据类型分类

JavaScript 语言的每一个值, 都属于某一种数据类型。JavaScript 的数据类型, 共有六种。(ES6 又新增了第七种 Symbol 类型的值和第八种 BigInt类型, 当前课程暂不涉及)

```
// null一般代表对象为“没有”  
// undefined一般代表数值为“没有”
```

`concat` 方法用于连接两个字符串，返回一个新字符串，不改变原字符串

```
1 var s1 = 'itbaizhan';  
2 var s2 = 'sxt';  
3  
4 s1.concat(s2) // "itbaizhansxt"  
5 s1 // "itbaizhan"
```

`substr` 方法用于从原字符串取出子字符串并返回，不改变原字符串，跟 `substring` 方法的作用相同

`substr` 方法的第一个参数是子字符串的开始位置（从0开始计算），第二个参数是子字符串的长度

```
1 'itbaizhan'.substr(2, 7); // baizhan
```

如果省略第二个参数，则表示子字符串一直到原字符串的结束

```
1 'itbaizhan'.substr(2) // "baizhan"
```

如果第一个参数是负数，表示倒数计算的字符位置。如果第二个参数是负数，将被自动转为0，因此会返回空字符串

```
1 'itbaizhan'.substr(-7) // "baizhan"  
2 'itbaizhan'.substr(4, -1) // ""
```

`indexOf` 方法用于确定一个字符串在另一个字符串中第一次出现的位置，返回结果是匹配开始的位置。如果返回 `-1`，就表示不匹配

```
1 'hello world'.indexOf('o') // 4  
2 'itbaizhan'.indexOf('sxt') // -1
```

`indexOf` 方法还可以接受第二个参数，表示从该位置开始向后匹配

```
1 'hello world'.indexOf('o', 6) // 7
```

`trim` 方法用于去除字符串两端的空格，返回一个新字符串，不改变原字符串

```
1 ' hello world '.trim()  
2 // "hello world"
```

该方法去除的不仅是空格，还包括制表符（`\t`）、换行符（`\n`）和回车符（`\r`）

```
1 '\r\nitbaizhan \t'.trim() // 'itbaizhan'
```

`split` 方法按照给定规则分割字符串，返回一个由分割出来的子字符串组成的数组

```
1 'it|sxt|baizhan'.split('|') // ["it", "sxt", "baizhan"]
```

如果分割规则为空字符串，则返回数组的成员是原字符串的每一个字符。

```
1 'a|b|c'.split('') // ["a", "|", "b", "|", "c"]
```

如果省略参数，则返回数组的唯一成员就是原字符串

```
1 'it|sxt|bz'.split() // [it|sxt|bz]
```

`split` 方法还可以接受第二个参数，限定返回数组的最大成员数。

```
1 'it|sxt|bz'.split('|', 0) // []  
2 'it|sxt|bz'.split('|', 1) // ["it"]  
3 'it|sxt|bz'.split('|', 2) // ["it", "sxt"]
```

`Array.isArray` 方法返回一个布尔值，表示参数是否为数组。它可以弥补 `typeof` 运算符的不足

```
1 var arr = ["尚学堂", 100, true];  
2 console.log(typeof arr); // object
```

```
1 var arr = ['sxt', 'baizhan', 'it'];  
2 Array.isArray(arr) // true
```

数组方法 `_push()`/`_pop()`



`push` 方法用于在数组的末端添加一个或多个元素，并返回添加新元素后的数组长度。注意，该方法会改变原数组

```
1 var arr = [];  
2 arr.push("尚学堂") // 1  
3 arr.push('itbaizhan') // 2  
4 arr.push(true, {}) // 4  
5 arr // [尚学堂, 'itbaizhan', true, {}]
```

`pop` 方法用于删除数组的最后一个元素，并返回该元素。注意，该方法会改变原数组

```
1 var arr = ['尚学堂', 'itbaizhan', 'WEB前端'];  
2  
3 arr.pop() // 'WEB前端'  
4 arr // ['尚学堂', 'itbaizhan']
```

shift 方法用于删除数组的第一个元素，并返回该元素。注意，该方法会改变原数组

```
1 var arr = ['尚学堂', 'itbaizhan', 'WEB前端'];
2
3 arr.shift() // '尚学堂'
4 arr // ['itbaizhan', 'WEB前端']
```

shift 方法可以遍历并清空一个数组

```
1 var list = [1, 2, 3, 4, 5, 6];
2 var item;
3
4 while (item = list.shift()) {
5   console.log(item);
6 }
7
8 list // []
```

unshift 方法用于在数组的第一个位置添加元素，并返回添加新元素后的数组长度。注意，该方法会改变原数组

```
1 var arr = ['尚学堂', 'itbaizhan', 'WEB前端'];
2
3 arr.unshift('baizhan'); // 4
4 arr // ['baizhan', '尚学堂', 'itbaizhan', 'WEB前端']
```

unshift 方法可以接受多个参数，这些参数都会添加到目标数组头部

```
1 var arr = ['尚学堂', 'itbaizhan'];
2 arr.unshift('WEB前端', 'baizhan') // 4
3 arr // ['WEB前端', 'baizhan', '尚学堂', 'itbaizhan']
```

join 方法以指定参数作为分隔符，将所有数组成员连接为一个字符串返回。如果不提供参数，默认用逗号分隔

```
1 var a = [1, 2, 3, 4];
2
3 a.join(' ') // '1 2 3 4'
4 a.join(' | ') // "1 | 2 | 3 | 4"
5 a.join() // "1,2,3,4"
```

如果数组成员是 **undefined** 或 **null** 或空位，会被转成空字符串

```
1 [undefined, null].join('#')
2 // '#'
3
4 ['a',, 'b'].join('--')
5 // 'a--b'
```

数组的 **join** 配合字符串的 **split** 可以实现数组与字符串的互换

数组的 **join** 配合字符串的 **split** 可以实现数组与字符串的互换

```
1 var arr = ["a","b","c"];
2 var myArr = arr.join("");
3 console.log(myArr);
4 console.log(myArr.split(""));
```